PORTAFOLIO

# Isabel Esther Rodríguez Alvarez

## Backend & AI Agent Engineer

**Python · FastAPI · Microsoft Azure · RAG · Multiagente · MCP**

Ingeniera de Software con 5 años de experiencia en desarrollo backend y agentes de IA. Diseño APIs y servicios cloud en Azure (Functions, Service Bus, SignalR, App Services, AI Search, Cosmos DB) y construyo agentes de IA que funcionan 24/7 en producción: desde sistemas multiagente para salud hasta asistentes que operan procesos reales de negocio. Este portafolio recoge cuatro proyectos con su arquitectura, decisiones técnicas y el trabajo de implementación.

Remoto · Cuba (horario de EE. UU. Este) · isabelestherr@gmail.com · LinkedIn · GitHub

**5 años**
de experiencia en backend e IA

**4**
casos de estudio reales

**64**
herramientas MCP propias en un solo agente

**+160k**
direcciones y lugares indexados en búsqueda

| | | |
|---|---|---|
| 1 | **VitalIA** | Plataforma de salud con IA |
| 2 | **Xuxa** | Asesora inmobiliaria con IA · ED Bienes Raíces (Panamá) |
| 3 | **Sage** | Agente de inventario y compras para restaurantes (Panamá) |
| 4 | **Proyectos de Backend** | Plataforma de movilidad: búsqueda geoespacial y mensajería en tiempo real |

## Habilidades

**Backend**
FastAPI · Django · Spring · Node.js · APIs REST · Arquitectura orientada a eventos · Clean Architecture / DDD

**Microsoft Azure**
Functions · Service Bus · SignalR · App Services · AI Search · Cosmos DB · Blob Storage · Key Vault · Container Registry · AI Foundry

**Agentes de IA**
Multiagente · RAG híbrido · MCP · Function calling · Memoria de agentes · Azure AI Foundry Agents · Azure OpenAI

### Datos

PostgreSQL

MongoDB / Cosmos DB · Redis

Búsqueda vectorial y geoespacial

### DevOps

Docker · Azure DevOps (CI/CD)

Application Insights

OpenTelemetry

> Por acuerdos de confidencialidad, no se muestra código fuente ni datos de clientes. Las capturas usan datos de prueba o anonimizados.

AGENTES DE IA + BACKEND

# VitalIA

## Plataforma de salud con IA

Junta médica multiagente, informes clínicos automáticos y chat médico en tiempo real.

Multiagente | Azure AI Foundry | FastAPI | Azure Functions | SignalR | Service Bus | FHIR

**3**
microservicios en Azure

**~9**
agentes especialistas en la junta médica

**8**
agentes de informes en paralelo

**~40k**
líneas en el motor de agentes

## Mi rol

> Arquitectura y desarrollo principal de los tres servicios de la plataforma: motor de agentes, orquestador y capa de tiempo real.

## Contexto y problema

VitalIA es una plataforma de salud que acompaña a profesionales y pacientes. Necesitaba analizar casos clínicos complejos con varias perspectivas médicas, generar informes periódicos a partir de datos clínicos estándar (FHIR) y ofrecer un asistente conversacional. Todo ello con respuestas trazables, porque en salud no basta con que la IA "suene bien", y con feedback en tiempo real mientras la IA trabaja.

## Solución

### Junta médica multiagente

Un agente coordinador analiza el caso y elige qué especialistas intervienen. Los especialistas proponen hipótesis, agentes críticos de cada especialidad las cuestionan, se refinan y un agente de consenso redacta un informe clínico con un nivel de confianza calculado.

### Informes clínicos por suscripción

Ocho agentes generan en paralelo alertas de signos vitales, tendencias poblacionales, estadísticas de condiciones y seguimiento, con la frecuencia y el volumen que define el plan del cliente.

### Chat médico con memoria

Asistente conversacional con historial persistente, títulos automáticos y sugerencias de preguntas personalizadas a partir de conversaciones anteriores.

### Tiempo real

El usuario ve el progreso del análisis (fase, ciclo y ronda) y las respuestas del agente a medida que se generan, a través de Azure SignalR.

## Arquitectura




Arquitectura general: tres servicios desacoplados que se comunican por HTTP, SignalR y Service Bus.

## Flujo de la junta médica multiagente

1. **Extracción clínica**
   Text Analytics for Health identifica entidades médicas del caso.
2. **Coordinador**
   Selecciona especialistas, nivel de urgencia y banderas rojas.
3. **Ronda 0 · Hipótesis**
   Los especialistas proponen diagnósticos en paralelo.
4. **Ronda 1 · Crítica**
   Agentes críticos por especialidad buscan debilidades y riesgos.
5. **Ronda 2 · Refinamiento**
   Los especialistas ajustan hipótesis y confianza.
6. **Consenso**
   Informe SOAP con ranking de hipótesis y confianza calculada.

## Tecnologías

| |
|---|
| **Agentes e IA** |
| Azure AI Foundry Agent Service, GPT-4o y GPT-4o-mini según el rol del agente, Azure OpenAI (embeddings), File Search con vector stores, Azure AI Language (Text Analytics for Health) |
| **Backend** |
| Python, FastAPI, Azure Functions (Python y Node.js), Pydantic, httpx asíncrono |
| **Datos** |
| Cosmos DB (API MongoDB), Azure AI Search (índice vectorial), Redis, Blob Storage |
| **Mensajería y tiempo real** |
| Azure SignalR Serverless, Azure Service Bus (topics y mensajes programados) |
| **Seguridad e integración** |
| Key Vault, Managed Identity, estándar FHIR R4/R5 |
| **DevOps** |
| Docker multi-stage, Azure Pipelines, Container Registry, App Service, Application Insights |
| **Arquitectura** |
| Clean Architecture, DDD, CQRS e inyección de dependencias |

## Implementación: retos y decisiones

**01** **Confianza calculada, no autoevaluada**

En lugar de pedirle al LLM que puntúe su propia seguridad, un algoritmo determinista pondera la consistencia entre especialistas, la calidad de la evidencia y la coherencia temporal. El resultado es auditable ronda a ronda.

### 02 Agentes críticos como "red team" clínico

Cada especialidad tiene un agente que propone y otro cuyo único objetivo es encontrar fallos en las hipótesis, lo que reduce el sesgo de confirmación.

### 03 Informes programados sin un cron por cliente

Cada plan de suscripción define su frecuencia. Al activarse, se programa un mensaje diferido en Service Bus; el worker lo procesa, lanza los agentes en paralelo y programa la siguiente ejecución.

### 04 Seguridad multi-cliente

Las credenciales FHIR de cada cliente se resuelven en Key Vault en tiempo de ejecución y nunca se guardan. Los servicios se autentican con Managed Identity.

### 05 SignalR fiable desde Python

El orquestador actúa como un cliente más del hub: conexión en un hilo dedicado, ping/pong propio, reconexión automática y re-suscripción a grupos y eventos tras cada caída.

### 06 Sin respuestas duplicadas al escalar

Con varias instancias, un mismo evento puede llegar dos veces. Un bloqueo en Redis con expiración evita llamar dos veces al modelo (que es lo caro) y duplicar mensajes.

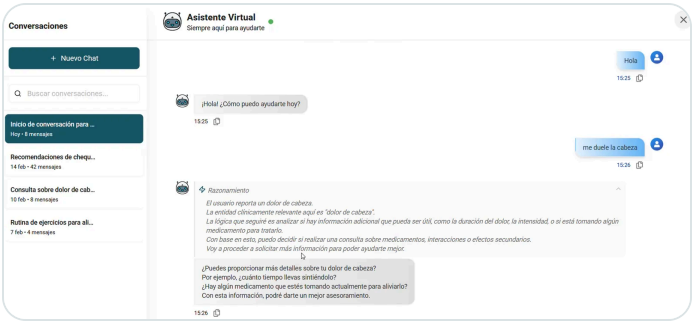
### 07 Memoria que no penaliza la latencia

Las conversaciones se indexan en AI Search con embeddings tras cinco minutos de inactividad, no en cada mensaje. Esa memoria alimenta después las sugerencias personalizadas.

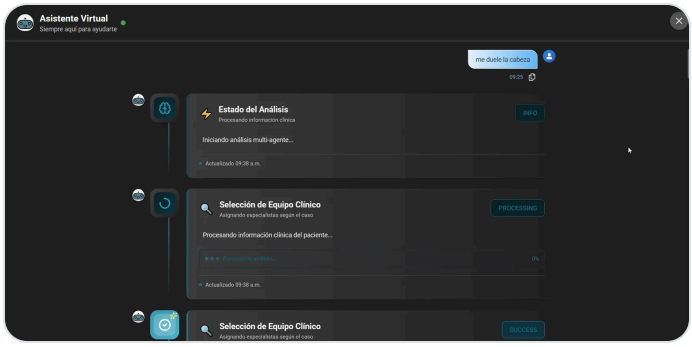
## 08 Refactor a Clean Architecture

Los agentes pasaron de código duplicado a una clase base común que concentra threads, reintentos y parseo robusto de JSON, eliminando cerca de mil líneas repetidas.

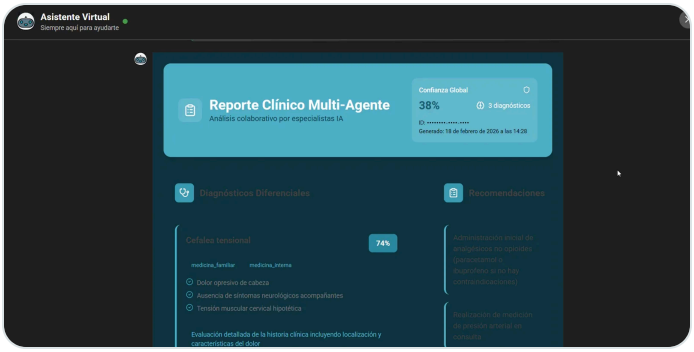
## Capturas



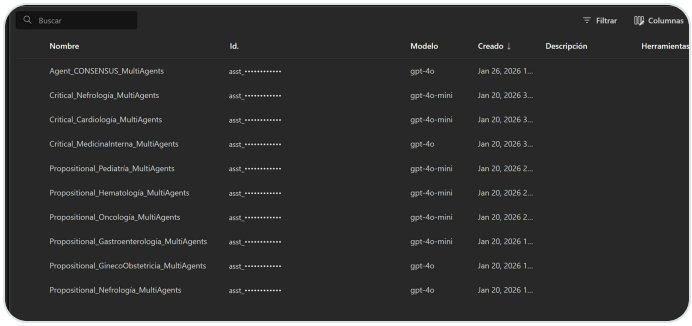

Chat médico respondiendo (datos de prueba)



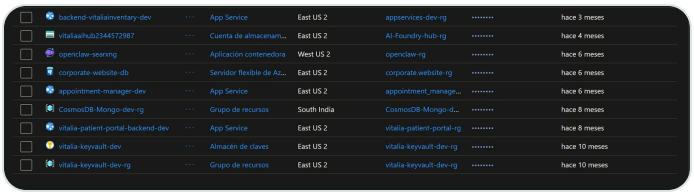

Análisis multiagente en curso: fase, ciclo y ronda




Reporte clínico final: diagnósticos diferenciales y confianza global




Azure AI Foundry: agentes propositivos, críticos y de consenso por especialidad




Recursos de la plataforma en Azure: App Services, Key Vault y almacenamiento

## Resultados

- Análisis clínico multiagente trazable, con historial de cada ronda consultable por API.
- Informes clínicos automáticos adaptados a cada plan de suscripción.
- Feedback en tiempo real del trabajo de la IA hacia el usuario.

- Despliegue continuo en Azure con contenedores y pipelines.

AGENTES DE IA

# Xuxa

## Asesora inmobiliaria con IA · ED Bienes Raíces (Panamá)

Agente 24/7 en WhatsApp, Telegram y web que busca propiedades, capta clientes y resuelve dudas legales.

WhatsApp · Telegram · Multiagente · RAG · MCP · Azure AI Search · App Service

**24/7**
atención en 3 canales

**2**
agentes: asesora + experta legal

**3**
servidores MCP propios

**DEV / PROD**
entornos separados con rollback

## Mi rol

> Diseño e implementación completa del agente sobre OpenClaw (gateway open source de agentes), extendido con plugins, servidores MCP, RAG legal e infraestructura en Azure.

## Contexto y problema

En ED Bienes Raíces todos los procesos de atención eran manuales: los clientes esperaban respuesta en horario laboral, los interesados se perdían sin registrarse y las preguntas legales (alquiler, compraventa, propiedad horizontal) consumían tiempo de los agentes.

## Solución

### Búsqueda de propiedades

Consulta el CRM inmobiliario AlterEstate con filtros de ciudad, sector, tipo, precio y operación (venta o alquiler), además de proyectos y unidades.

### Captación automática de clientes

Cuando detecta interés real (pide visita, precio o dice "me interesa") solicita los datos de contacto y registra el lead en el CRM etiquetado por canal.

### Lex, experta legal

Subagente especializado en legislación inmobiliaria de Panamá. Responde con RAG sobre leyes oficiales, citando ley, artículo y fuente.

### Back-office por roles

Clientes, agentes y administradores tienen herramientas distintas: Excel, base de datos, gestión de usuarios y, para administración, correo y calendario.

### Documentos y voz

Genera PDFs con la marca de la empresa, transcribe notas de voz y ofrece un chat web público de demostración.

## Arquitectura




Un gateway de agentes en contenedor recibe los mensajes de todos los canales y orquesta dos agentes y sus herramientas.

### Recorrido de un mensaje

1

**Recepción**

Llega texto o nota de voz; el audio se transcribe con Azure Whisper.

2

**Identificación**

En el primer mensaje se identifica al usuario y su rol en la base de datos.

3

**Razonamiento**

El modelo decide qué herramientas usar según la intención y el rol.

4

**Acción**

Busca en el CRM, registra el lead, consulta a Lex o genera un documento.

5

**Respuesta**

Entrega por el mismo canal, con adjuntos si hace falta.

6

**Memoria**

La conversación se guarda y se compacta para sesiones largas.

## Tecnologías

| | |
|---|---|
| **IA** | |
| Azure AI Foundry (model-router), Azure Whisper, embeddings, RAG híbrido con reranker semántico | |
| **Plataforma de agentes** | |
| OpenClaw (TypeScript, Node.js 22) con plugins y parches propios | |
| **Herramientas** | |
| Servidores MCP propios (base de datos, usuarios, Excel), plugin para el CRM con 8 herramientas, generador de PDF | |
| **Datos** | |
| Cosmos DB for MongoDB, Azure AI Search, Blob Storage | |
| **Integraciones** | |
| CRM AlterEstate (REST), Microsoft Graph, WhatsApp, Telegram | |

**Infraestructura**

App Service para contenedores (DEV y PROD), Container Registry, Azure Files, Container Apps, Managed Identity

## Implementación: retos y decisiones

### 01 Sesiones de WhatsApp que se "robaban"

Cargar dependencias desde un disco de red bloqueaba el proceso unos 12 segundos; la conexión perdía latidos y WhatsApp retomaba la sesión. Se separó el estado en disco local de la configuración compartida, eliminando el bloqueo.

### 02 RAG legal fiable

Las leyes se trocean por artículo, las búsquedas usan un umbral de relevancia y hasta tres reformulaciones antes de admitir que no hay información. La cita de la fuente y el aviso legal son obligatorios.

### 03 Multiagente sin bucles

Xuxa solo delega preguntas jurídicas y Lex trabaja en una sesión aislada con tiempo límite, con reglas explícitas para evitar delegaciones circulares.

### 04 Mantener un fork vivo

Cinco actualizaciones del framework base conservando todas las personalizaciones. Las extensiones se hicieron como plugins para reducir conflictos.

### 05 Defensa frente a alucinaciones

Registro de documentos recientes para recuperar archivos cuando el modelo "inventa" rutas, y reglas estrictas para trabajar con Excel y adjuntos.

### 06 Seguridad por capas

Roles resueltos en base de datos, perfiles de herramientas por agente, servidores MCP en solo lectura para el chat público y protección frente a peticiones a destinos no permitidos.

## Capturas

Búsqueda de propiedades pedida por nota de voz, transcrita y consultada en el CRM

Xuxa delega en Lex, la experta legal, una consulta sobre desalojo

PDF con la marca de la empresa generado por el agente

Panel de gestión de usuarios y roles

Portal de Azure: App Services de DEV y PROD, Foundry y servicios utilizados

Panel de administración: modelo y canales conectados (Telegram y WhatsApp)

## Resultados

- Atención 24/7 en WhatsApp, Telegram y web, sustituyendo un flujo que era completamente manual.
- Registro automático de interesados en el CRM, etiquetados por canal de origen.
- Respuestas legales con fuente verificable para los clientes de la inmobiliaria.
- Despliegues controlados en DEV y PROD con copia previa y marcha atrás documentada.




AGENTES DE IA + BACKEND

# Sage

## Agente de inventario y compras para restaurantes (Panamá)

El personal gestiona inventario, compras y recepción de mercancía conversando con un agente.

MCP · Telegram · Cosmos DB · Roles y aprobaciones · App Service · Azure Files

**64**
herramientas MCP propias

**5**
roles con permisos dinámicos

**105/105**
pruebas de permisos superadas

**2**
alertas programadas al día

## Mi rol

> Diseño e implementación sobre OpenClaw: toda la lógica de negocio en servidores MCP propios, extensiones del gateway, modelo de datos e infraestructura en Azure.

## Contexto y problema

Una cadena de restaurantes controlaba inventario y compras con hojas de cálculo. Las compras no tenían aprobaciones trazables, la mercancía se recibía sin evidencia y el stock real se desviaba del teórico. El personal de cocina y almacén trabaja con el móvil, no con un ERP.

## Solución

### Inventario auditable

Cada cambio es un movimiento (compra, venta, producción, merma, devolución, transferencia, ajuste) con quién, cuándo y por qué. El stock semanal se recalcula desde los movimientos.

### Compras con aprobación por roles

Solicitudes numeradas con estados y límites de monto por rol y sucursal. Las compras directas avisan si el precio cambió respecto a la última vez.

### Recepción en dos pasos

El almacenero registra lo recibido con calidad y foto; otra persona aprueba o rechaza cada línea. Solo lo aprobado mueve el inventario.

### Alertas de reposición

Dos veces al día el agente revisa el stock y avisa por Telegram, distinguiendo entre stock bajo y agotado.

### Unidades del mundo real

Convierte sacos, quintales, cajas o libras a la unidad base según el producto y el proveedor. Si falta un dato, pregunta en vez de adivinar.

### Gestión y demo

Panel web de usuarios y permisos, notas de voz, lectura y generación de Excel, y un chat público de demostración con datos anonimizados.

## Arquitectura




El agente no toca la base de datos directamente: toda operación pasa por herramientas MCP que validan rol, permisos y datos.

## Ciclo de una solicitud de compra

**1**

### Pendiente

Se crea con número automático y sucursal.

**2**

### Aprobada

La aprueba un rol con límite de monto suficiente.

**3**

### En tránsito

El proveedor la envía; puede recibirse por partes.

**4**

### Recepción

Registro con calidad y foto de evidencia.

**5**

### Revisión

Otra persona aprueba o rechaza cada línea.

**6**

### Recibida

El inventario se actualiza y se recalcula la semana.

## Tecnologías

| IA |
| --- |
| Azure AI Foundry (model-router), Azure Whisper para notas de voz |

| Plataforma de agentes |
| --- |
| OpenClaw (TypeScript, Node.js) con extensiones propias: chat público, API de usuarios y panel web |
| **Lógica de negocio** |
| Servidores MCP propios en Node.js: inventario y compras, usuarios y permisos, Excel |
| **Datos** |
| Cosmos DB for MongoDB (vCore) con validación de esquema, base de demostración anonimizada |
| **Infraestructura** |
| App Service para contenedores, Container Registry, Azure Files (evidencias y configuración) |
| **Calidad** |
| Tests del motor de unidades y suite de extremo a extremo de permisos |

## Implementación: retos y decisiones

### 01 No confiar ciegamente en el modelo

El rol se lee de la base de datos, las escrituras exigen una confirmación firmada ligada a los datos exactos y la base de datos está fijada por configuración. Nació de un incidente real en el que el modelo improvisó una estructura de datos.

### 02 Separación de funciones

Quien recibe la mercancía no puede aprobarla, y quien propone una sustitución de producto no puede confirmarla. Se garantiza en las herramientas, no en el prompt.

### 03 Respuestas lentas por concurrencia

El bot tardaba hasta dos minutos con mensajes seguidos. Se diagnosticó una condición de carrera en el inicio de sesión del framework y se corrigió con reintentos y espera exponencial.

### 04 Persistencia en contenedores

Las fotos de evidencia caían en almacenamiento efímero. Se movieron a un recurso compartido de Azure Files organizado por pedido.

### 05 Inventario como libro mayor

Los movimientos son la fuente de verdad y el stock semanal es una vista recalculada en cadena, incluso cuando se registra un movimiento de una semana pasada.

### 06 Demo pública segura

Agente separado, base anonimizada, solo herramientas de lectura, límites de peticiones y CORS. Los bloqueos del filtro de contenido se responden como un rechazo normal.

## Capturas

Compra por chat: aviso de cambio de precio y confirmación antes de registrar

Recepción: foto de evidencia y aprobación de la línea

Alerta de reposición programada

Conversión de unidades: quintales, sacos, libras y kilos

Panel de permisos por rol

Panel de usuarios (teléfonos ocultos)

Panel de administración: usuarios, roles y permisos por herramienta

## Resultados

- Compras y recepciones con trazabilidad completa y aprobación humana en cada paso crítico.
- Alertas automáticas de reposición sin revisar hojas de cálculo.

- Permisos reasignables desde un panel web, con efecto inmediato en el agente.
- Demo pública para enseñar el producto sin exponer datos reales.

BACKEND

# Proyectos de Backend

## Plataforma de movilidad: búsqueda geoespacial y mensajería en tiempo real

Servicios backend para una app de transporte que conecta pasajeros y conductores.

Python · FastAPI · Azure AI Search · Service Bus · Azure SignalR · Azure Functions · Azure DevOps

**2**
servicios en producción

**3**
colas de Service Bus

**4**
etapas en el pipeline de despliegue

**3**
modos de envío: usuario, grupo o todos

### Mi rol

> Evolución del motor de búsqueda de direcciones en un servicio existente, y desarrollo completo del servicio de mensajería en tiempo real.

### Contexto y problema

Una app de movilidad necesita que el pasajero encuentre su destino en segundos escribiendo texto libre, y que pasajeros y conductores intercambien eventos al instante aunque la conexión móvil sea inestable.

### Solución

**Motor de búsqueda de direcciones**

Devuelve lugares y direcciones ordenados por relevancia, popularidad y cercanía, con geocodificación inversa, cálculo de rutas y alta de comercios.

**Procesamiento por colas**

Las búsquedas entran por eventos y se procesan de forma asíncrona con Service Bus, sin bloquear la recepción de mensajes.

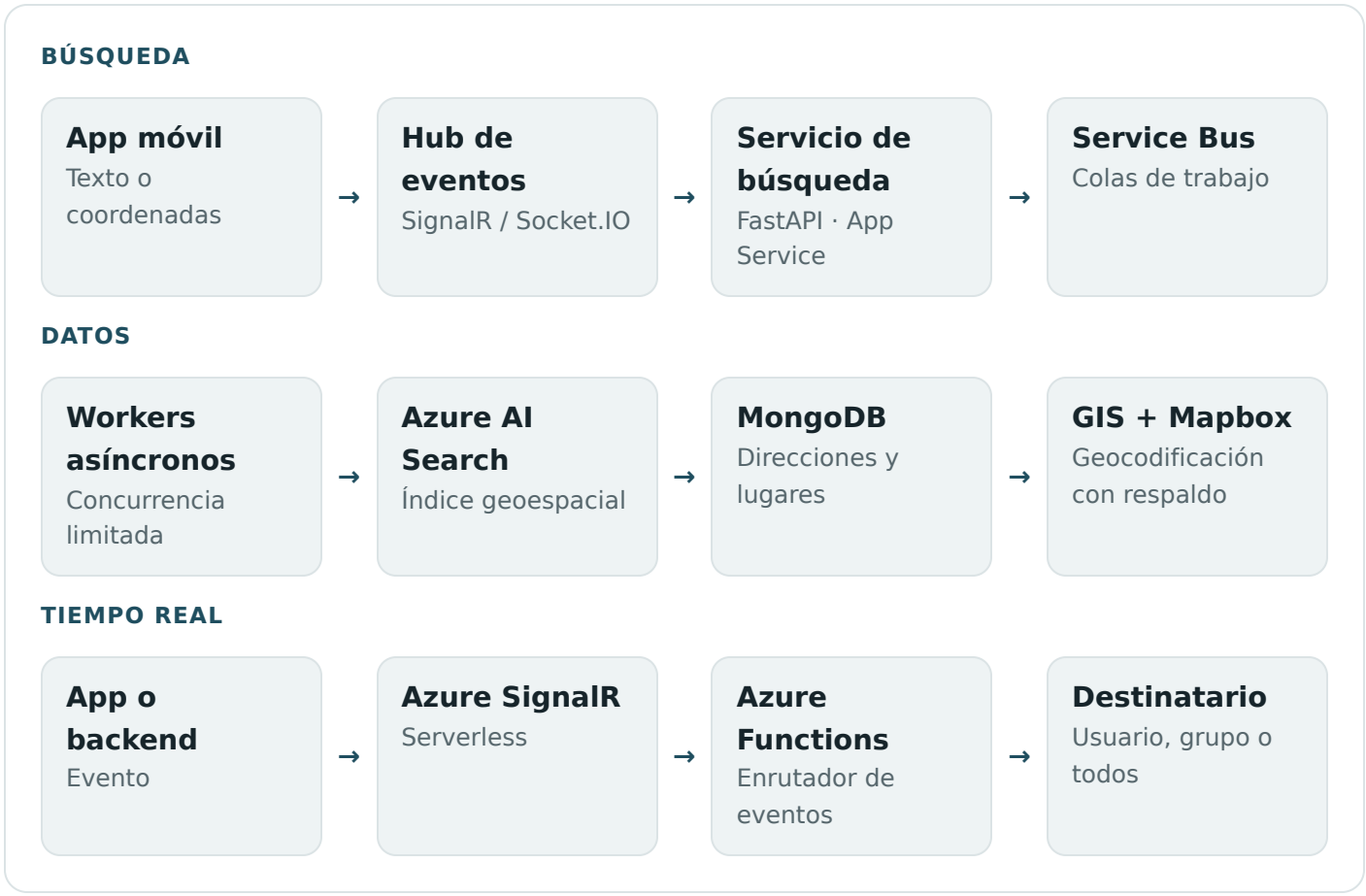
### Mensajería en tiempo real serverless

Azure SignalR y Azure Functions enrutan eventos entre usuarios concretos, grupos o todos, sin un servidor dedicado.

### Contratos de eventos como artefacto

La especificación AsyncAPI se genera, se valida y se publica automáticamente en los repositorios que la consumen.

## Arquitectura




Dos servicios complementarios: búsqueda geoespacial por eventos y un hub de mensajería serverless.

### Recorrido de una búsqueda

1
**Evento**
La app envía el texto buscado por el hub de tiempo real.

2
**Validación**
Se valida el mensaje y se identifica al usuario.

3
**Cola**
Se encola en Service Bus y el receptor queda libre al instante.

4
**Consulta**
Dos búsquedas en paralelo: lugares y direcciones, con filtro por distancia.

5
**Fusión**
Se combinan por relevancia y popularidad.

6
**Respuesta**
Se devuelve solo a ese usuario, comprimida según la versión de la app.

## Tecnologías

| |
|---|
| **Búsqueda** |
| Python 3.11, FastAPI, Azure AI Search (índice geoespacial, analizador en español, perfiles de puntuación) |
| **Mensajería** |
| Azure Service Bus, Azure SignalR Serverless, Socket.IO |
| **Serverless** |
| Azure Functions en Node.js (modelo v4), Azurite para desarrollo local |
| **Datos e integraciones** |
| MongoDB, Blob Storage, API GIS, Mapbox (geocodificación y rutas) |
| **DevOps** |
| Docker, Azure DevOps (pipelines de 4 etapas), Container Registry, App Service, AsyncAPI |

**Observabilidad**

OpenTelemetry, Application Insights, métricas propias de cola, health checks con auto-reparación

## Implementación: retos y decisiones

### 01 Desacoplar recepción y procesamiento

El receptor de eventos se bloqueaba con búsquedas lentas. Se introdujo Service Bus con un productor de larga vida y un consumidor asíncrono con concurrencia controlada.

### 02 Relevancia de búsqueda

Lugares y direcciones se consultan en paralelo y se fusionan con criterios distintos: relevancia para lugares y popularidad para direcciones, con umbral mínimo configurable.

### 03 Intenciones declarativas

Reglas en un archivo de configuración convierten búsquedas como "restaurante" o "aeropuerto" en filtros por categoría, ajustables sin desplegar código.

### 04 Conexiones inestables

Reconexión en dos capas, latido propio y un health check que avisa a Azure para reciclar la instancia si la conexión no se recupera.

### 05 Hub de mensajería sin servidor

SignalR en modo Serverless con Functions: los clientes invocan eventos que atienden funciones, y el nombre del evento se decide en tiempo de ejecución, así que se añaden eventos nuevos sin desplegar.

### 06 Observabilidad con coste controlado

Muestreo de telemetría y métricas propias de tiempo en cola y fallos de procesamiento.

## Capturas

Búsqueda de direcciones en la app: sugerencias mientras se escribe

Pipeline de Azure DevOps: validación de APIs, despliegue y publicación de contratos

Contrato AsyncAPI de los eventos, generado y validado en el pipeline

Índices de Azure AI Search: más de 160.000 direcciones y lugares en producción

Function App de tiempo real: triggers de SignalR y HTTP

## Resultados

- Búsqueda de direcciones rápida y relevante sin bloquear el canal de eventos.
- Mensajería en tiempo real que escala a cero y admite eventos nuevos sin desplegar.
- Contratos de API siempre sincronizados entre backend, frontend y tiempo real.